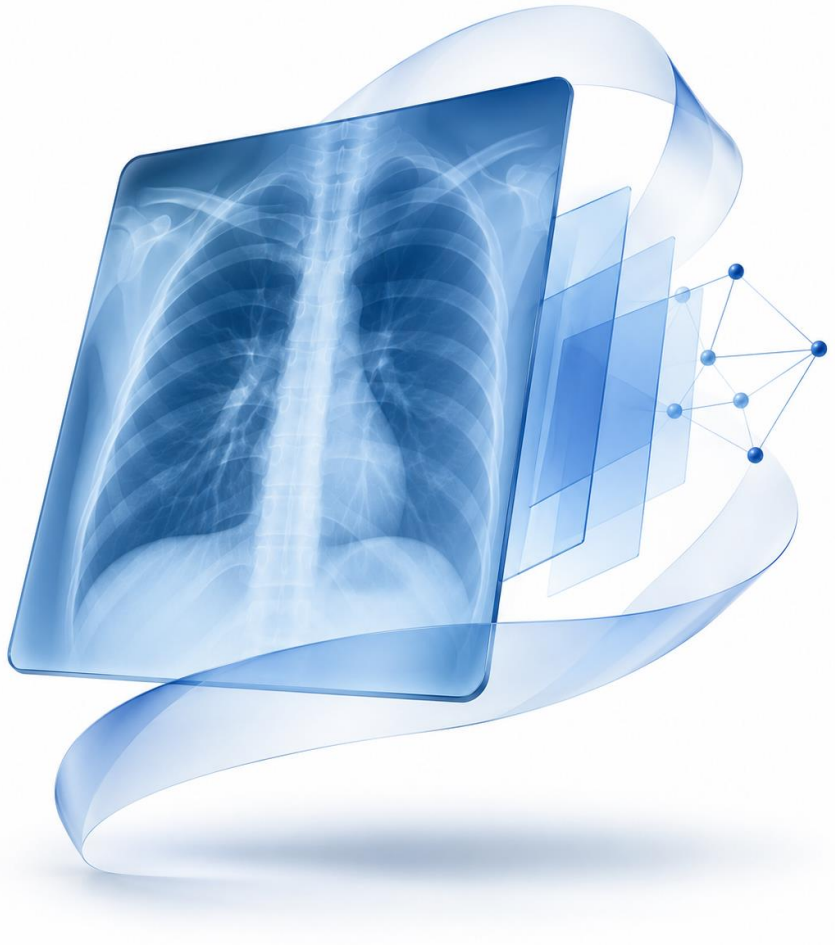


تصنيف صور أشعة الصدر

Pneumonia Classification with PyTorch

د .محمد الحبيب

Postdoc Researcher

Computer Vision & Deep Learning

Intelligent remote sensing research center

Chinese Academy of Agricultural Sciences (CAAS), China

مشروعنا اليوم

تصنيف Chest X-rayإلى فئتين باستخدام PyTorch



Input: Chest X-ray

NORMAL

صورة مصنّفة طبيعية في Dataset

PNEUMONIA

صورة مصنّفة بالتهاب رئوي في Dataset

نجهّز الصور وندرّب Classifier، ثم نختبره على صور جديدة

ما معنى Pneumonia؟

عدوى في الرئة قد تسببها Bacteria أو Viruses أو Fungi

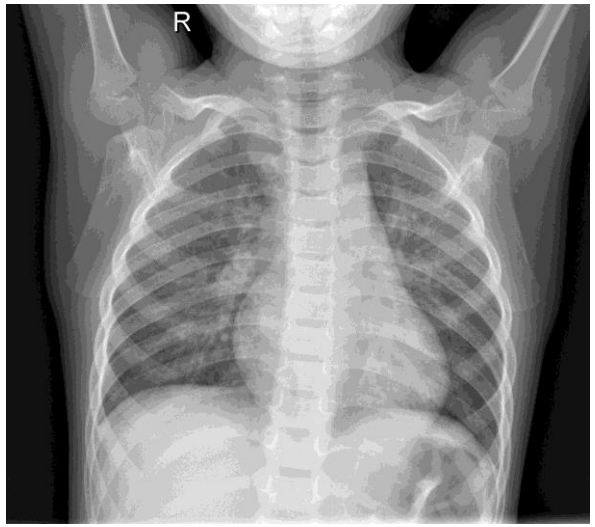


قد تمتلئ Alveoli بالسوائل أو القيح، فيصعب تبادل الأكسجين والتنفس.

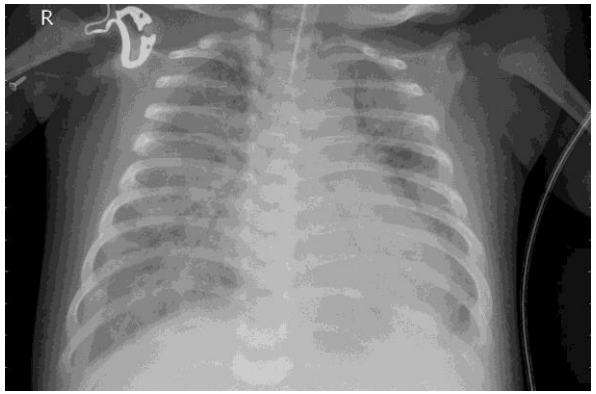
تشخيص المرض يجمع الأعراض والفحص والاختبارات، ومنها Chest X-ray.

البيانات التي سنستخدمها

Chest X-Ray Images (Pneumonia)



NORMAL



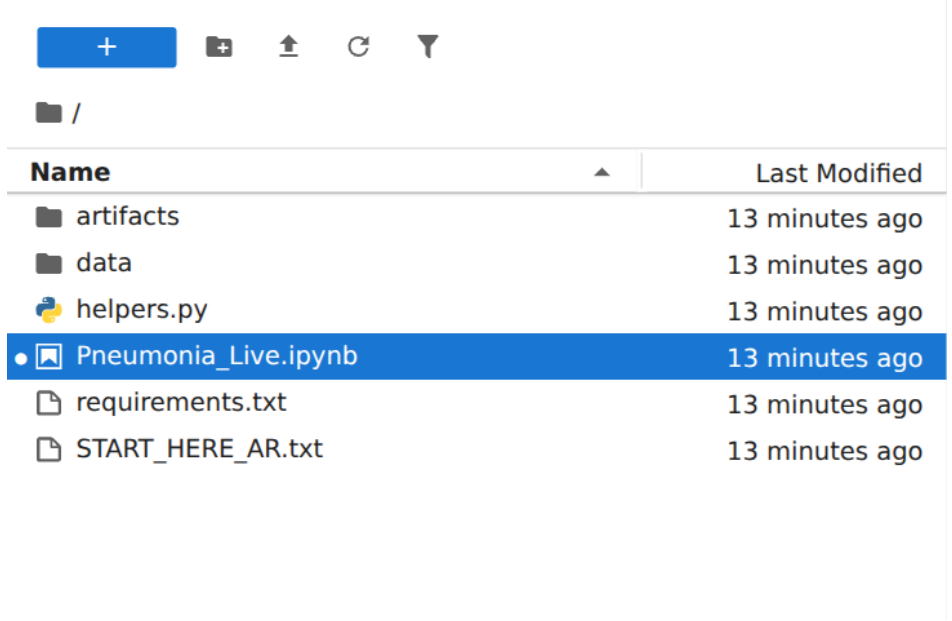
PNEUMONIA

أطفال من سنة إلى خمس سنوات
Guangzhou، الصين
الفئتان هما Labels للصور
نسخة الكلاس جاهزة في الحزمة

تنزيل المصدر الأصلي من Kaggle

kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia

مجلد المشروع الذي سنفتحه



`Pneumonia_Live.ipynb`

الخلايا التي سنكتب ونشغّل منها المشروع

`helpers.py`

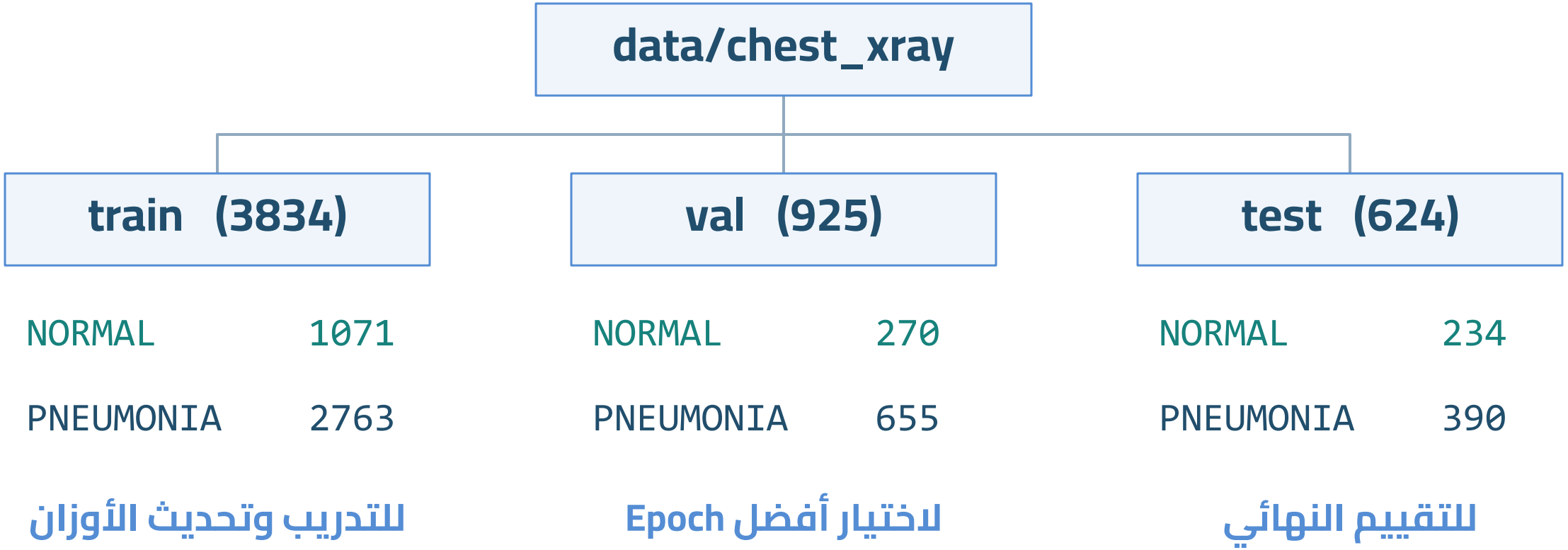
دوال الرسم، ومنها Grad-CAM

`data/` **`artifacts/`**

الصور في data، والأوزان في artifacts

داخل مجلد الصور

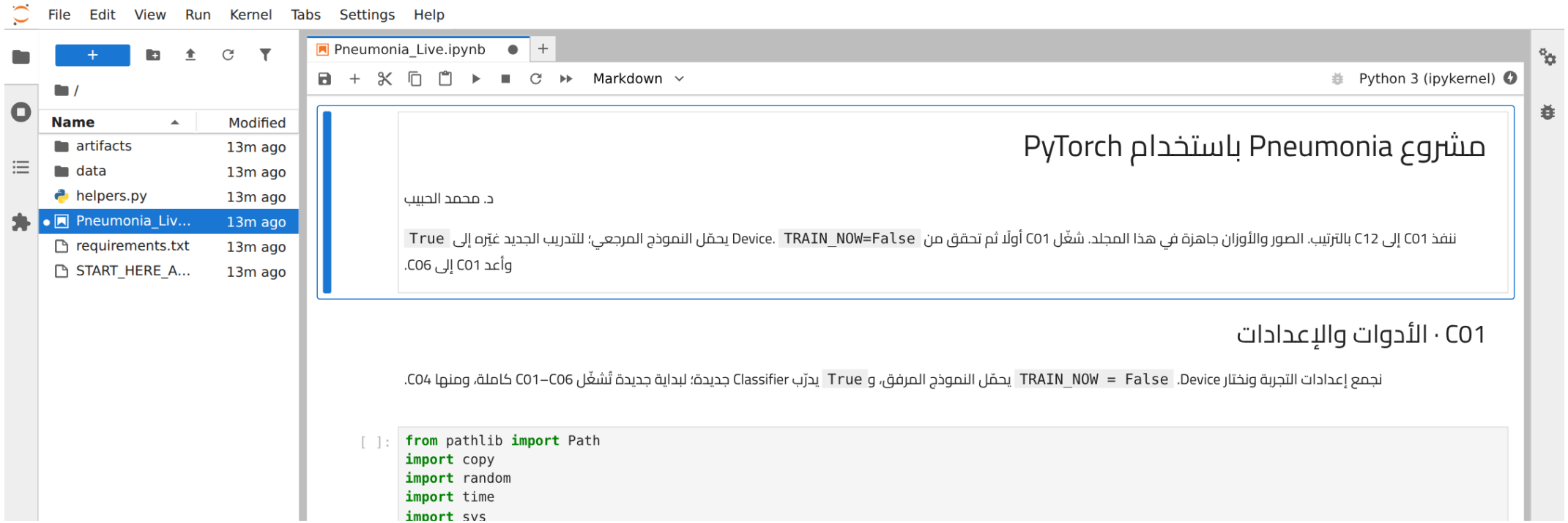
5383صورة في نسخة الحصة، موزعة على ثلاثة مجلدات



داخل كل مجلد NORMAL: وPNEUMONIA، والصور بصيغة JPEG

البداية داخل Jupyter

نفتح Pneumonia_Live.ipynb من مجلد المشروع



تشغيل الخلية والانتقال إلى التالية، بالترتيب من C01

Shift + Enter

C01: إعدادات التجربة

نحدد المقاس وحجم Batch وعدد Epochs قبل قراءة الصور

```
IMAGE_SIZE = 224
```
مقاس الصورة بعد Resize

```
BATCH_SIZE = 32
```
عدد الصور في الدفعة الواحدة

```
EPOCHS = 3
```
عدد مرات المرور على Train

```
LEARNING_RATE = 0.001
```
حجم خطوة التحديث في Adam

نستخدم CUDA عند توفرها، وإلا CPU

```
DATA_DIR = Path("data/chest_xray")
```

C02: تجهيز الصورة وقراءة Label

ImageFolder يقرأ الصور، و transform يجهّزها عند الاستخدام



[3, 224, 224]

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])


train_data = datasets.ImageFolder(
    DATA_DIR / "train", transform=transform
)
```

NORMAL = 0 **PNEUMONIA = 1**

C02: تجهيز القيم قبل MobileNetV2

نستخدم Preprocessing المناسب لأوزان كل إطار

Keras (TensorFlow)

```
from tensorflow import keras
mobilenet = (
    keras.applications.mobilenet_v2)
preprocess = mobilenet.preprocess_input
```

Pixel range: [-1, 1]

PyTorch

```
MEAN = [0.485, 0.456, 0.406]
STD = [0.229, 0.224, 0.225]
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])
```

ToTensor ثم Normalize لكل Channel

Resize وحدها لا تكفي لتجهيز قيم الصور للنموذج

C02–C03: قراءة الصور وتجميع Batch

أسماء المجلدات تحدد Labels في المثالين

Keras (TensorFlow)

```
load_images = (
    keras.utils
    .image_dataset_from_directory)
train_ds = load_images(
    DATA_DIR / "train",
    image_size=(224, 224),
    batch_size=BATCH_SIZE,
    label_mode="int", shuffle=True)
train_ds = train_ds.map(
    lambda x, y: (preprocess(x), y))
```

Images: [N, 224, 224, 3]

PyTorch

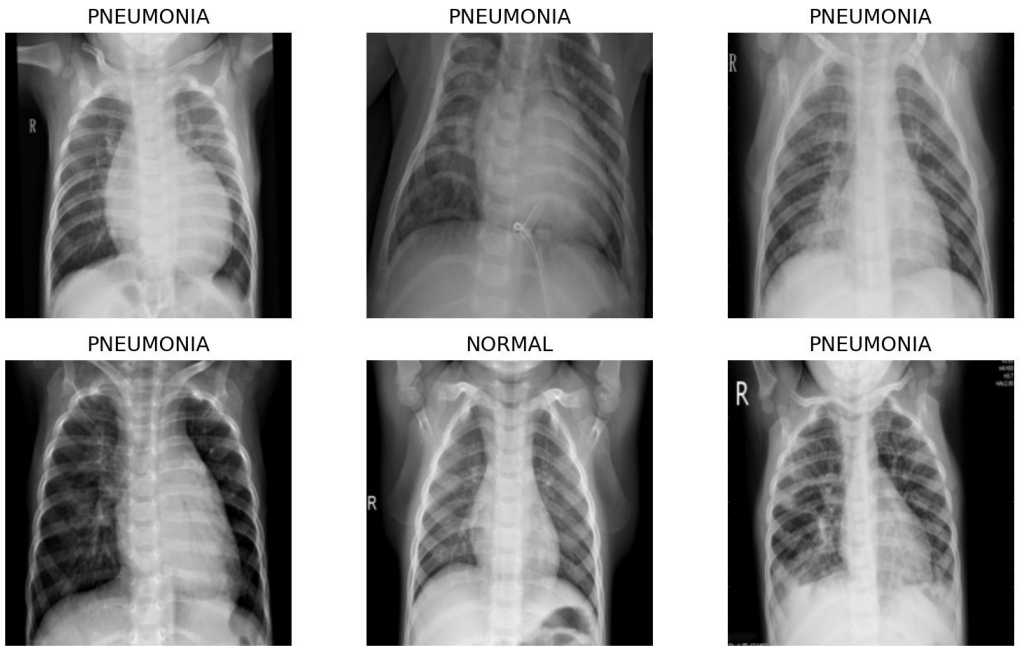
```
train_data = datasets.ImageFolder(
    DATA_DIR / "train",
    transform=transform)
train_loader = DataLoader(
    train_data, batch_size=BATCH_SIZE,
    shuffle=True, num_workers=0)
```

Images: [N, 3, 224, 224]

نستخدم Labels رقمية، ونخلط Train فقط

C03: تجميع الصور في Batches

Dataset يوفّر الصور، و DataLoader يجمعها في دفعات



images

`[32, 3, 224, 224]`

labels

`[32]`

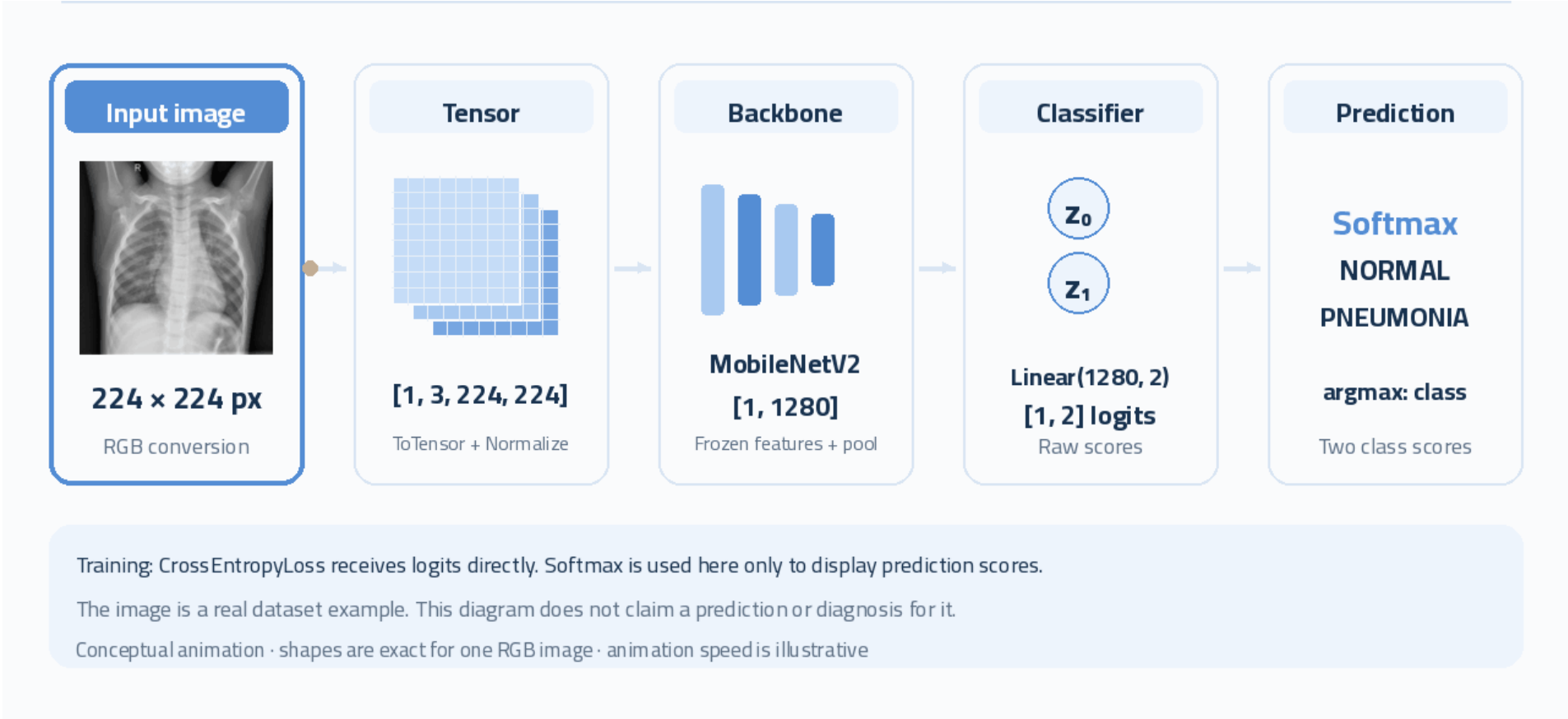
نخلط Train فقط

المعروض 6 صور للمعاينة، أما Batch الكاملة ففيها 32 صورة

```
Batches: Train = 120     Validation = 29     Test = 20
```

04C: النموذج الذي سنستخدمه

MobileNetV2 تستخرج Features، Classifierg تتعلم التمييز بين الفئتين



نثبت Backbone، ونجعل Linear الأخيرة تخرج درجتين: NORMAL وPNEUMONIA

C04: تجميد Backbone

نحتفظ بما تعلمته MobileNetV2 على ImageNet

Keras (TensorFlow)

```
base = mobilenet.MobileNetV2(
    weights="imagenet",
    include_top=False,
    pooling="avg",
    input_shape=(224, 224, 3))
base.trainable = False
```

Forward: base(inputs, training=False) في

PyTorch

```
model = models.mobilenet_v2(
    weights=models.MobileNet_V2_Weights
                  .IMAGENET1K_V2)
for parameter in (
    model.features.parameters()):
    parameter.requires_grad = False
```

model.train(training): بعد run_epoch داخل

```
model.features.eval()
```

requires_grad: Gradients | eval(): BatchNorm / Dropout

C04: المخرجات وقياس الخطأ

في هذه المقارنة نستخدم درجتين خامًا للفئتين في الإطارين

Keras (TensorFlow)

```
inputs = keras.Input(shape=(224, 224, 3))
x = base(inputs, training=False)
x = keras.layers.Dropout(0.2)(x)
logits = keras.layers.Dense(2)(x)
model = keras.Model(inputs, logits)
loss_fn = (
    keras.losses
    .SparseCategoricalCrossentropy(
        from_logits=True))
```

Dense(2) بدون Activation

PyTorch

```
model.classifier[-1] = nn.Linear(
    model.last_channel,
    len(class_names))
model = model.to(device)
loss_fn = nn.CrossEntropyLoss()
```

CrossEntropyLoss تستقبل Logits مباشرة

Softmax عند عرض Scores في Prediction

Linear: 1280 Features إلى 2 Logits

Logits: [N, 2] | Labels: [N] | Classes: 0, 1

الرامي وأرقام نموذجنا

نراقب وزنًا واحدًا أثناء خطوة تدريب باستخدام Adam





1 Forward

Weight	-0.023899
Gradient	not yet
Loss	1.1325
P(PNEUMONIA)	32.22%

Prediction: NORMAL

أرقام فعلية من Classifier، والرامي تشبيه بصري

ما الذي تغيّر في هذه الخطوة؟

تحدّثت أوزان Classifier، ونتتبّع هنا وزنًا واحدًا فقط

Adam: lr = 0.001 | Tracked weight: W[1, 0]

Value	Before	After backward	After Adam
Weight	-0.023899	-0.023899	**-0.022899**
Loss	1.1325	1.1325	**0.8974**
P(PNEUMONIA)	32.22%	32.22%	**40.76%**

```
Gradient = (0.3222204 - 1) * 0.0303217 ~ -0.020551
```

Weight change = +0.001000

التصنيف ما زال NORMAL، رغم انخفاض Loss

C05: ماذا يحدث لكل Batch؟

داخل run_epoch: الصور x، والإجابات الصحيحة y

```
optimizer.zero_grad(set_to_none=True)
```
نمسح Gradients السابقة

```
logits = model(x)
```
النموذج يحسب درجتين لكل صورة

```
loss = loss_fn(logits, y)
```
نحسب الخطأ باستخدام Labels الصحيحة

```
loss.backward()
```
نحسب Gradients

```
optimizer.step()
```
Adam يحدّث أوزان Classifier

Validation تستخدم النموذج لقياس الأداء دون تحديث الأوزان

C04–C06: تشغيل التدريب في الإطارين

نحدد Optimizer وLoss، ثم نكرر Training وValidation

Keras (TensorFlow)

```
model.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=LEARNING_RATE),
    loss=loss_fn,
    metrics=["accuracy"])
model.fit(train_ds,
    validation_data=val_ds,
    epochs=EPOCHS)
```

fit تدير التدريب والتقييم عبر Epochs

PyTorch

```
optimizer = torch.optim.Adam(
    model.classifier.parameters(),
    lr=LEARNING_RATE)

for epoch in range(1, EPOCHS + 1):
    run_epoch(train_loader,
              training=True)
    run_epoch(val_loader,
              training=False)
```

run_epoch تنفّذ الخطوات التي كتبناها في C05

run_epoch دالتنا في Notebook، وليست دالة جاهزة من PyTorch

06C: تدريب جديد أم نموذج جاهز؟

TRAIN_NOW يحدد المسار الذي تنفذه هذه الخلية

```
TRAIN_NOW = True
```

تحديث Classifier على بيانات Train
Progress لـTraining و Validation
نتائج خاصة بالتجربة الجديدة

```
TRAIN_NOW = False
```

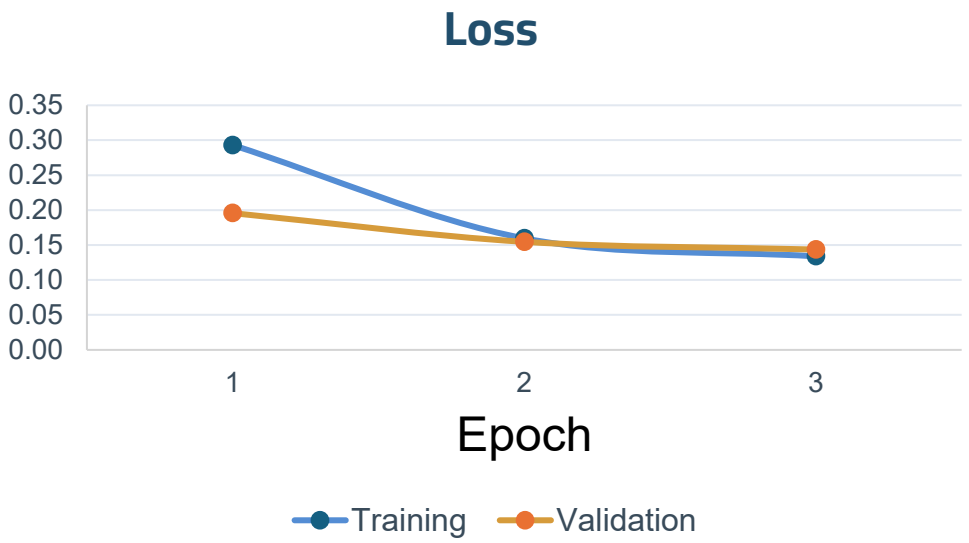
تحميل النموذج المدرب المرفق
ظهور رسالة Loading ثم Loaded
لا يحدث Training في هذا المسار

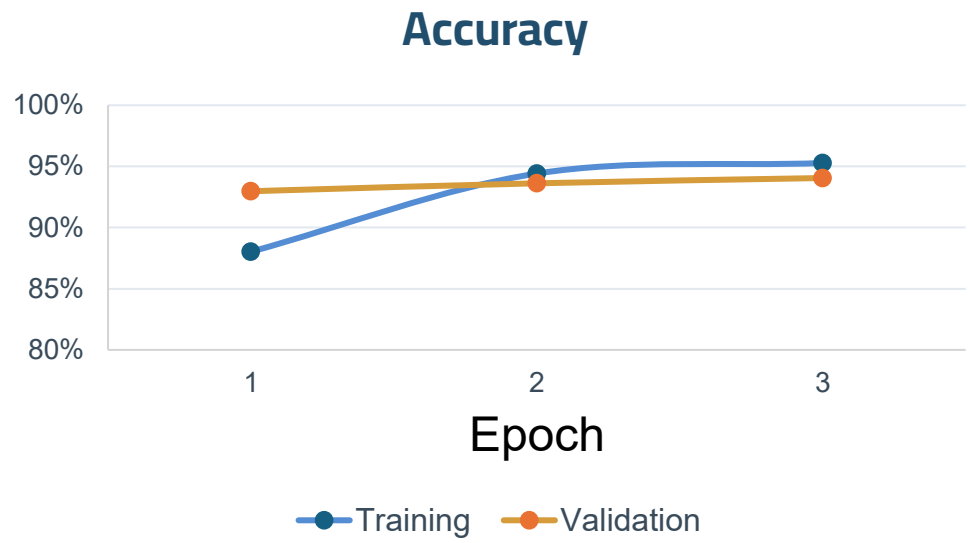
لبداية جديدة: تشغيل 06C–01C بالترتيب يعيد إنشاء Model و Optimizer في 04C.

07C: منحنيات التدريب

من history في مشروعنا؛ هذه نتائج النموذج المرجعي بعد 3 Epochs

```
plot_history(history, best_epoch)
```





نختار أقل Validation Loss. تراجع Validation مع تحسّن Train يستحق المراجعة.

08C: التقييم على جميع صور Test

624 صورة، Confusion Matrixg للنموذج المرجعي

True label	Pred: NORMAL	Pred: PNEUMONIA
NORMAL	TN = 101	FP = 133
PNEUMONIA	FN = 8	TP = 382

صنّف النموذج 133 صورة NORMAL على أنها PNEUMONIA.

Accuracy = 77.4% **Recall = 97.9%**

Recall = 382 / (382 + 8)

Specificity = 43.2%

C09: حفظ النموذج وإعادة تحميله

نحفظ ما نحتاجه لاستخدام النموذج على صورة جديدة

artifacts/pneumonia_classroom.pt

```
Weights      Class names      Image size      MEAN / STD
```

```
torch.save(checkpoint, "artifacts/pneumonia_classroom.pt")
```

نحمّل الملف في deployed_model قبل Prediction

حفظ أسماء Classes وإعدادات الصورة يحافظ على معنى المخرجات

11C–10C: تجربة صورة واحدة

predict تعالج الصورة، ثم show_prediction تعرض النتيجة



True: PNEUMONIA

Class	Score
NORMAL	16.6%
PNEUMONIA	83.4%

Pred: PNEUMONIA

نضيف Batch dimension

[3,224,224]

unsqueeze(0)

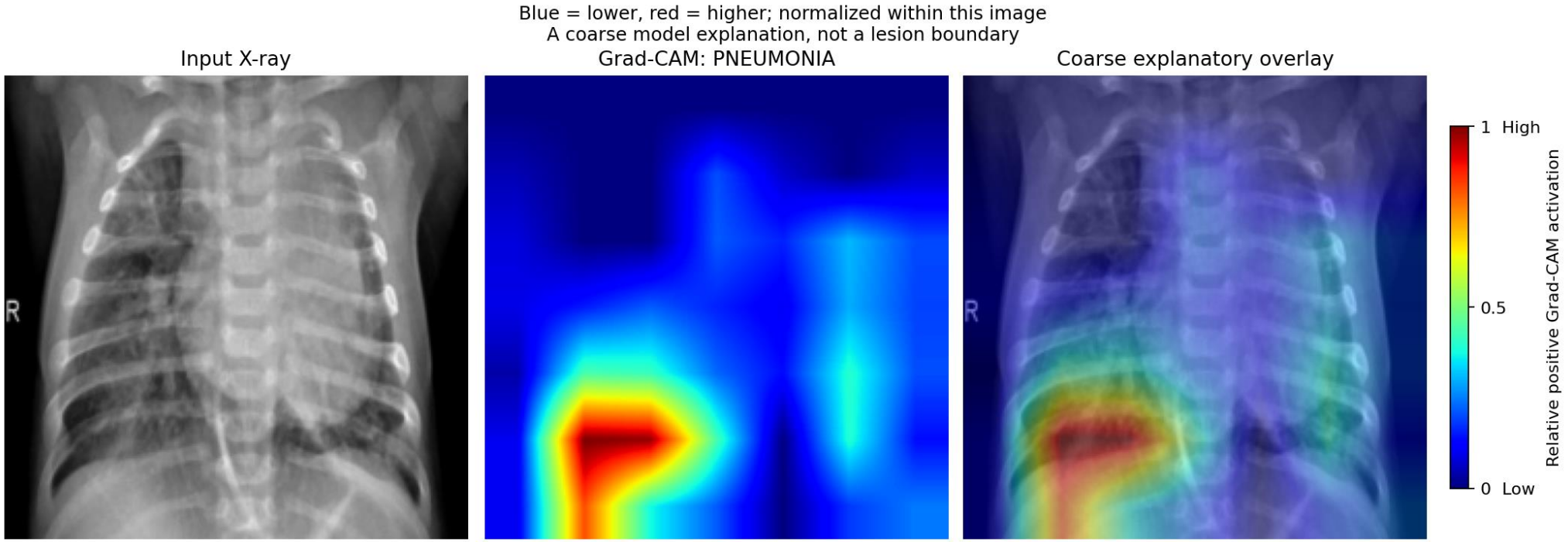
[1,3,224,224]

مثال Test: الصورة 234

الدرجة تعبّر عن توزيع Scores داخل النموذج؛ ليست يقيناً طبياً.

12C: قراءة Grad-CAM

show_gradcam تفسّر Prediction للصورة والنموذج نفسيهما



أزرق: مساهمة منخفضة أحمر: مساهمة أعلى داخل الخريطة نفسها

الألوان لا تحدد حدود المرض، ولا تثبت صحة Prediction.

تحدي الحصة Fine-tuning :

C13–C14تستخدمان نموذجنا ودالة run_epoch نفسها

الجزء	المشروع الأساسي	التحدي
التحديث	Classifierفقط	آخر Feature block + Classifier
Learning Rate	0.001	0.0001
البيانات	Trainالكاملة أو النموذج المرجعي	10 Batchesمن Train = 320صورة
المقارنة	نتائج النموذج الأساسي	Validationقبل التحديث وبعده

النتيجة قد تتحسن أو تتراجع؛ المطلوب تفسير ما حدث

هذه تجربة قصيرة على جزء من Train، وليست Epochكاملة

ساعة التطبيق داخل الحصة

- Pneumonia_Practice.ipynb نفس C01–C12 في شرح المدرّب

العمل المطلوب	الوقت
إكمال TODO01–05 وتشغيل C01–C12	16:30–17:05
TODO06 ثم Fine-tuning ومقارنة Validation	17:05–17:20
C15: تذكرة الخروج وحفظ التسليم	17:20–17:30

التسليم Notebook: باسمك Pneumonia_Class_Submission.zip +

ماذا غيّرت؟ ماذا حدث في Validation؟ وماذا تعني Grad-CAM؟

Home work | تصنيف ثلاث فئات

- Pneumonia_Homework_3Class.ipynb بعد الحصة

NORMAL



BACTERIAL



VIRAL



نقسم PNEUMONIA بحسب virus g bacteria في أسماء الملفات

نجهّز البيانات، ونستخدم 3 Outputs، ثم نقيّم كل Class